

22 — Level 1 rejects DP structurally: nothing can cover the hotel load

Proves · Level 1 rejecting structurally: no combination can cover the hotel load at all.

Mechanics this scenario turns on

- Validation runs **before** Level 1. A plant that cannot carry its load is rejected there, with a different message and a different code path from the Level 1 rejections.
- The shaft generator is filled before any auxiliary starts (the main engine is already turning), and its output is a load **on** that main engine — which is why the ME figure exceeds propulsion. SG capacity scales with the number of running MEs.

Panels described below · The plant · The response · Why the branch order matters

Anything not described here — the mechanics above name the step that produced it; [00-ORIENTATION](#) Part 6 has the full number-to-step index.

Trust · characterisation snapshot, generated from the code. It detects change; it does NOT prove correctness. Figures marked *pending reference verification* have never been checked against anything outside the application.

Read after · scenario 20.

The fourth and last rejection branch — [ExplainFor](#) 's fallback, reached when **only** structural rejections fired: no PTI problem, no aux overload, no power shortfall, simply no arrangement of the installed machinery that works at all.

The plant

ME 2 × 5 000 · no SG · AE 2 × 800 · DP: thrust 2 000, hotel 5 000

The thrust is easy. The hotel load is the problem: 5 000 kW against an aux fleet of 1 600 kW and no shaft generator. Every candidate fails the *hotel must be fully covered* rule, which is counted as [Structural](#) — so none of the three more specific branches ever fires and the fallback wins.

Transit (hotel 800) is comfortably feasible, so again the failure is DP-only and validation lets it through.

The response

400

No feasible engine configuration: no engine configuration can cover the DP demand.

Check the engine capacities, engine counts and the power demands for this mode.

This is the vaguest of the four messages, and deliberately so: when every candidate died for a structural reason there is no single dominant cause to point at, so the advice lists the three inputs worth re-reading rather than guessing.

Why the branch order matters

`ExplainFor` checks in this order: battery PTI gate → power shortfall → aux overload → fallback. A plant can trip several at once; the user is told about the most specific one. Scenarios 09, 20, 21 and 22 pin one branch each, and `Level1RejectionDiagnosticsTests` pins the exact wording plus the precedence.

Takeaway: a 400 from this calculator always names the mode and always suggests something to change. That is a contract, and it now has four scenarios holding it.

KSailCalc manual test scenarios — calculation walkthrough · regenerated 2026-08-07 · numbers verified against commit 559aef2 and unchanged by the backend/client refactors (18 golden snapshots byte-identical)